

Level 1: Hybrid Environment and Ingestion (Code-First)

Exercise 1: Query on Non-Optimized Table (Diagnostic)

The Country Manager for **Germany** urgently needs to review the transactions of **March 12, 2022** .

Task:

- » 1. Write the query that joins (JOIN) transactions and companies.
- » 2. Filter the results by the indicated date and the country "Germany".
- » 3. Without executing the query, perform a "Dry Run" (cost audit).

It cost 11.17MB



The screenshot shows a SQL query editor with a menu bar (File, Edit, Run, Tools) and a toolbar (Run, Save, Schedule, Share). The query text is as follows:

```
1 -- Level 1: Hybrid Environment and Ingestion (Code-First)
2 -- Exercise 1: Query on Non-Optimized Table (Diagnostic)
3 -- The Country Manager for Germany urgently needs to review the transactions of March 12, 2022 .
4 -- 1. Write the query that joins ( JOIN) transactions and companies.
5 -- 2. Filter the results by the indicated date and the country "Germany".
6 -- » 3. Without executing the query, perform a "Dry Run" (cost audit).
7 select *
8 from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean` AS tr
9 join `sprint3-analytics-mahnaz.sprint3_silver.companies_clean` AS co
10 on tr.business_id=co.company_id
11 where co.country= 'Germany' AND date(tr.timestamp)= '2022-03-12';
```

At the bottom of the editor, a status bar indicates:  This query will process 11.17 MB when run.

Task:

Step 1: Generating Recent Data (Mocking Data)

Create an intermediate table called `sprint3_silver.transactions_recent` from the `sprint3_silver.transactions_clean` table . Your goal is to keep all the columns , but replace the original timestamp with a new one, randomly generated so that it falls within the last 50 days .

```
create or replace table `sprint3-analytics-mahnaz.sprint3_silver.transactions_recent` AS
select *
```

```
    except(timestamp),
    TIMESTAMP_SUB(
    CURRENT_TIMESTAMP(),
    INTERVAL CAST(FLOOR(RAND() * 50) AS INT64) DAY
    ) AS timestamp
```

```
from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean`;
```

```
14 -- Exercise 2: Re-architecture and Storage Optimization (Partition & Cluster)
15 -- Step 1: Generating Recent Data ( Mocking Data)
16 -- Create an intermediate table called sprint3_silver.transactions_recent from the sprint3_silver.
17 -- transactions_clean table .
18 -- Your goal is to keep all the columns , but replace the original timestamp with a new one, randomly generated
19 -- so that it falls within the last 50 days .
20
21
22 create or replace table `sprint3-analytics-mahnaz.sprint3_silver.transactions_recent` AS
23 select *
24     except(timestamp),
25     TIMESTAMP_SUB(
26     CURRENT_TIMESTAMP(),
27     INTERVAL CAST(FLOOR(RAND() * 50) AS INT64) DAY
28     ) AS timestamp
29 from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean`;
30
```

✓ This script will process 21.56 MB when run.

sprint3-analytics-mahnaz / Datasets / sprint3_silver / Tables / transactions_recent						
transactions_...						
Query Chat Open in Share Copy Refresh						
Schema	Details	Preview	Insights	Lineage	Data profile	Data Quality
Row	product_ids	user_id	lat	longitude	timestamp	
1	81	1	40.7174604...	-73.9960221...	2026-09-22 10:32:44.126216 UTC	
	94					
	95					
	66					
	64					
2	17	1	40.7036804...	-74.0080139...	2026-09-16 10:32:44.126216 UTC	
	66					
	3					
3	83	1	40.7094684...	-74.0063754...	2026-09-21 10:32:44.126216 UTC	
	78					
	79					
	12					
	49					
4	26	1	40.7093088...	-74.0115485...	2026-09-21 10:32:44.126216 UTC	
	20					
	79					
5	7	1	40.7218733...	-73.9965615...	2026-09-08 10:32:44.126216 UTC	

Step 2: Creating the Optimized Table (Partitioning & Clustering)

Now, create your final table `sprint3_gold.fact_transactions_optimized` from the recent data you just generated in `transactions_recent`. You need to construct the DDL statement to configure the table with the following physical storage strategies:

- » **Partitioning** : Divides the table by the date in the `DATE(timestamp)` field. This will allow queries that filter by day (`WHERE date = ...`) to only read the necessary partition.
- » **Clustering** : Sorts the data within each partition by `business_id`. This will dramatically speed up filters by company and intersections (`JOINS`) with the companies dimension.

```
create or replace table `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`  
partition by date(timestamp)  
cluster by business_id  
AS  
select *  
from `sprint3-analytics-mahnaz.sprint3_silver.transactions_recent`;
```

The screenshot displays a SQL query execution environment. The query, which includes comments explaining partitioning and clustering, has been executed successfully. A green checkmark and the text "Query completed" are visible. Below the query, a notification bar states: "This statement created a new table named fact_transactions_optimized." with a "Go to table" button. The interface also shows a "Recent" jobs list on the left and tabs for "Job information", "Results", "Execution details", and "Execution graph" at the bottom.

```
--  
28  
29 -- Step 2: Creating the Optimized Table (Partitioning & Clustering)  
30 -- Now, create your final table sprint3_gold.fact_transactions_optimized from the recent data you just  
   generated in transactions_recent.  
31 -- You need to construct the DDL statement to configure the table with thenfollowing physical storage strategies  
32 --Partitioning : Divides the table by the date in the DATE(timestamp) field. This will allow queries that filter by  
   day ( WHERE date = ...) to only read the necessary partition.  
33 -- Clustering : Sorts the data within each partition by business_id. This will dramatically speed up filters by  
   company and intersections ( JOINS) with the companies dimension.  
34 create or replace table `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`  
35 partition by date(timestamp)  
36 cluster by business_id  
37 AS  
38 select *  
39 from `sprint3-analytics-mahnaz.sprint3_silver.transactions_recent`;  
40
```

✓ Query completed

Using on-demand processing quota

Recent Query results Chat View all jobs Save results Open in

12:50 create or replace ...

12:50 create or replace ...

12:49

Job information Results Execution details Execution graph

This statement created a new table named fact_transactions_optimized. Go to table

" Details " of the new fact_transactions_optimized table where you can see that it is partitioned and clustered correctly .

sprint3-analytics-mahnaz / Datasets / sprint3_gold / Tables / fact_transactions_optimized

☆ fact_transacti...

🔍 Query

💬 Chat

🔗 Open in ▾

👤 Share ▾

📄 Copy

⋮

🔄 Refresh

📘 This is a partitioned table. [Learn more](#)

Dismiss

Schema

Details

Preview

Insights

Lineage

Data profile

Data Quality

Default collation

Default rounding mode

Case insensitive

Description

Labels

Primary key(s)

Tags

Table type

Partitioned by

Partitioned on field

Partition expiry

Partition filter

Clustered by

Partitioned

DAY

timestamp

Partitions do not expire

Not required

business_id

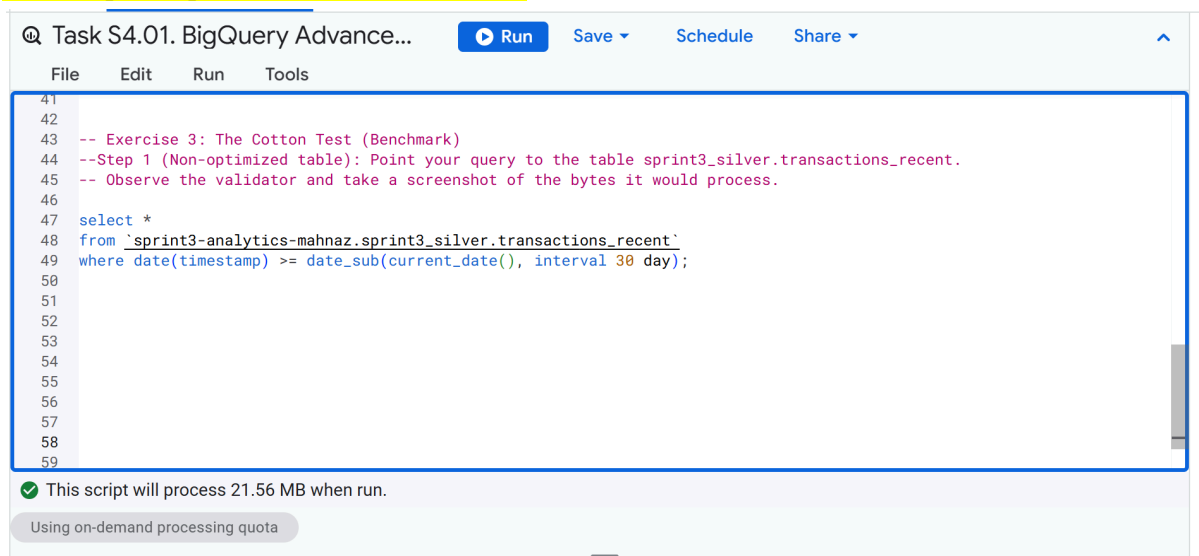
Exercise 3: The Cotton Test (Benchmark)

✦ Task:

The goal is clear and straightforward: **apply exactly the same query to two different tables and compare their computational cost**.

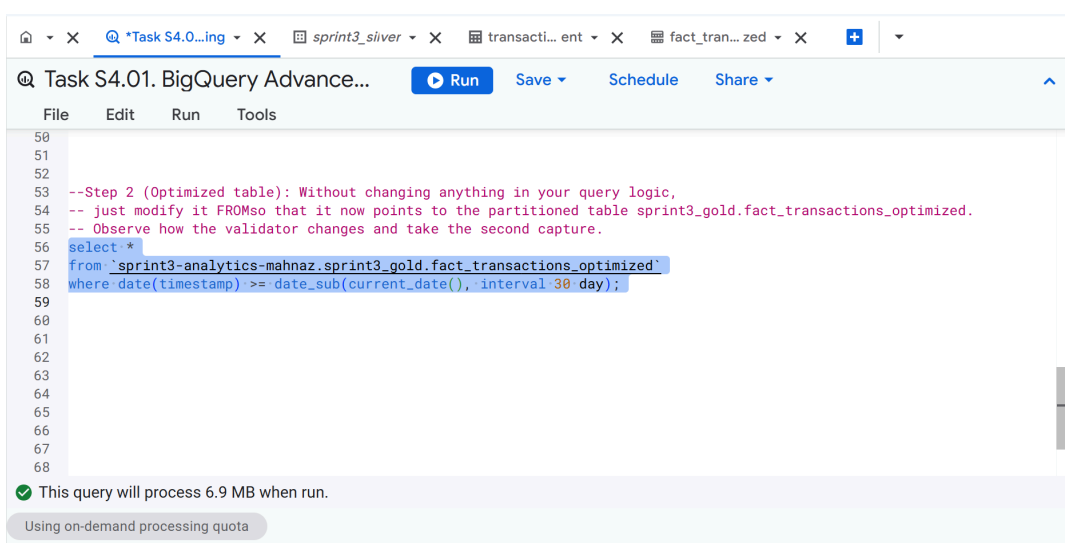
» **Step 1 (Non-optimized table):** Point your query to the table `sprint3_silver.transactions_recent`. Observe the validator and take a screenshot of the bytes it would process.

This script will process 21.56 MB when run.



```
Task S4.01. BigQuery Advance... Run Save Schedule Share
File Edit Run Tools
41
42
43 -- Exercise 3: The Cotton Test (Benchmark)
44 -- Step 1 (Non-optimized table): Point your query to the table sprint3_silver.transactions_recent.
45 -- Observe the validator and take a screenshot of the bytes it would process.
46
47 select *
48 from `sprint3-analytics-mahnaz.sprint3_silver.transactions_recent`
49 where date(timestamp) >= date_sub(current_date(), interval 30 day);
50
51
52
53
54
55
56
57
58
59
✓ This script will process 21.56 MB when run.
Using on-demand processing quota
```

» **Step 2 (Optimized table):** Without changing anything in your query logic, just modify it FROM so that it now points to the partitioned table `sprint3_gold.fact_transactions_optimized`. Observe how the validator changes and take the second capture.



```
*Task S4.0...ing sprint3_silver transactions... ent fact_tran... zed
Task S4.01. BigQuery Advance... Run Save Schedule Share
File Edit Run Tools
50
51
52
53 -- Step 2 (Optimized table): Without changing anything in your query logic,
54 -- just modify it FROM so that it now points to the partitioned table sprint3_gold.fact_transactions_optimized.
55 -- Observe how the validator changes and take the second capture.
56
57 select *
58 from `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`
59 where date(timestamp) >= date_sub(current_date(), interval 30 day);
60
61
62
63
64
65
66
67
68
✓ This query will process 6.9 MB when run.
Using on-demand processing quota
```

After applying partitioning to the optimized fact table, the estimated bytes processed decreased from 21.56 MB to 6.9 MB. This represents approximately 68% savings in data processed. The improvement is due to partition pruning, because BigQuery only scans the partitions corresponding to the last 30 days instead of scanning the whole table.

$$\text{Savings \%} = ((21.56 - 6.9) / 21.56) \times 100 \\ \approx 68\%$$

Exercise 4: Smart Caching (Materialized Views)

Scenario:

The CEO has a dashboard that shows " **Total Sales per Day** ." He (and 50 other managers) refresh this chart constantly.

Problem :

If you use a normal query or Standard View, every time someone refreshes the page, BigQuery recalculates the sum of millions of rows. It's slow, and you're paying for the same calculation over and over again.

Solution :

Use a **Materialized View** . This technology caches the calculation *result and only updates it if new data is entered (incremental)*.

Task:

» Create a **Materialized View** named `sprint3_gold.mv_daily_sales` that displays **Total Sales by Day** .

```
create materialized view `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`  
AS  
select date(timestamp) AS date, sum(amount) AS total_sales_per_day  
from `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`  
group by date(timestamp);
```

```

60
61
62 -- Exercise 4: Smart Caching (Materialized Views)
63 -- Create a Materialized View named sprint3_gold.mv_daily_salesthat displays Total Sales by Day .
64 create materialized view `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`
65 AS
66 select date(timestamp) AS date, sum(amount) AS total_sales_per_day
67 from `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`
68 group by date(timestamp);
69
70
71
72
73
74
75
76
77

```

✓ Query completed

Recent
Query results
Chat
View all jobs
Save results
Open in

16:42
create materializ...
12:50
Level 1: Hybrid ...
12:50

Job information
Results
Execution details
Execution graph

This statement created a new materialised view named mv_daily_sales.
Go to materialised view

```

60
61
62 -- Exercise 4: Smart Caching (Materialized Views)
63 -- Create a Materialized View named sprint3_gold.mv_daily_salesthat displays Total Sales by Day .
64 create materialized view `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`
65 AS
66 select date(timestamp) AS date, sum(amount) AS total_sales_per_day
67 from `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`
68 group by date(timestamp);
69
70 select *
71 from `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`
72 order by date;
73
74

```

✓ This query will process 800 B when run.

Using on-demand processing quota

Recent
Query results
Chat
View all jobs
Save results
Open in

16:47
select * from `spr...
16:42
create materializ...
12:50
Level 1: Hybrid ...
12:50
create or replace ...
12:49

Job information
Results
Visualisation
JSON
Execution details
Execution graph

Row	date	total_sales_per_d...
1	2026-08-16	514402.1900000...
2	2026-08-17	524453.9600000...
3	2026-08-18	516969.3500000...
4	2026-08-19	513898.0300000...
5	2026-08-20	525018.7800000...
6	2026-08-21	535364.4200000...

Results per page: 50 1 – 50 of 50

Level 2: Advanced Analytical SQL

Exercise 1: VIP Customer Profiling (Aggregated Metrics with CTEs)

Scenario:

The Marketing team wants to analyze the behavior of our best customers to design the strategy for next year. We define "VIPs" as those with a cumulative spend of more than €500. They need a report that, for each VIP, shows their name, contact and their purchasing pattern: how many times they have purchased, how much they spend on average and what their record purchase was.

Task:

» 1. Create a CTE called VIP_Stats that groups by user and calculates :

- Total Expense (SUM).
- The Quantity of Transactions (COUNT).
- The Average Ticket (AVG), rounded to 2 decimal places.
- The Maximum Purchase (MAX).
- Filter: Keep only those whose Total Expense is > 500.

» Cross the CTE with users_combined to obtain the personal data.

Exit Requirements:

- Columns: user_id, full_name, email, num_purchases, average_ticket, max_purchase, total_spent.
- Sorted by total_spent descending.

```
81 --The Average Ticket (AVG), rounded to 2 decimal places.
82 --The Maximum Purchase (MAX).
83 --Filter: Keep only those whose Total Expense is > 500.
84 with VIP_Stats AS (
85     select user_id,
86            sum(amount) AS Total_Expense,
87            count(transaction_id) AS num_purchases,
88            round(avg(amount), 2) AS Average_Ticket,
89            max(amount) AS Maximum_Purchase
90     from `sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized`
91     group by user_id
92     having Total_Expense > 500
93 )
94 -- Cross the CTE with users_combined to obtain the personal data.
95 --Columns: user_id, full_name, email, num_purchases, average_ticket, max_purchase, total_spent.
96 -- Sorted by total_spent descending.
97 select us.user_id, concat(us.name, ' ', us.surname) AS full_name, us.email,
98        vi.num_purchases, vi.Average_Ticket, vi.Maximum_Purchase, vi.Total_Expense
99
100 from `sprint3-analytics-mahnaz.sprint3_silver.users_combined` as us
101 join VIP_Stats as vi
102 on us.user_id = vi.user_id
103 order by vi.Total_Expense desc;
104
105
```

✓ This query will process 5.13 MB when run.

88
89
90
91
92
93
94
95

```
-- round(avg(amount), 2) AS Average_Ticket,  
-- max(amount) AS Maximum_Purchase  
-- from sprint3-analytics-mahnaz.sprint3_gold.fact_transactions_optimized  
-- group by user_id  
-- having Total_Expense>500  
-- )  
-- Cross the CTE with users_combined to obtain the personal data.  
-- Columns: user_id, full_name, email, num_purchases, average_ticket, max_purchase, total_spent
```

✔ This query will process 5.13 MB when run.

Using on-demand processing quota

Query results

Chat

View all jobs

Save results

Open in

Job information

Results

Visualisation

JSON

Execution details

Execution graph

Row	user_id	full_name	email	num_purchases	Average_Ticket
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35
2	318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	66	522.59
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	78	410.46
5	185	Molly Gilliam	donec@outlook.couk	110	275.75
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	81	300.67
9	349	Yxxcjk Fnceqbkmxt	yxxcjk.fnceqbkmxt@example.c...	53	443.48

Exercise 2: Trend Analysis (Window Functions on Views)

Scenario:

Finance Management wants to monitor daily "sales velocity." They need a report that compares each day's performance against the previous day to detect sudden drops or spikes in growth (Day-over-Day Growth).

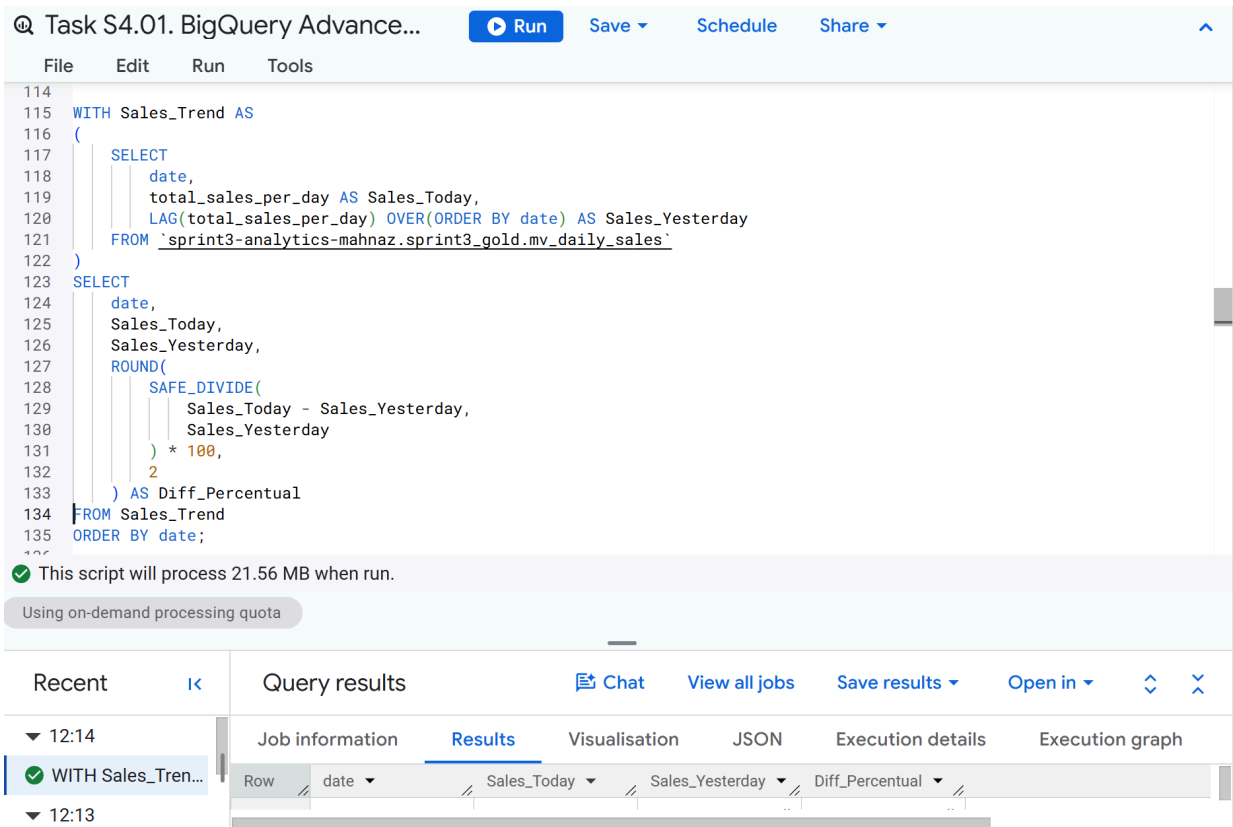
Optimization Strategy:

For this calculation you do NOT need to go to the original fact table or recalculate sums. As a good Data Engineer, you should use the Materialized View `sprint3_gold.mv_daily_sales` that you created in the previous exercise. This ensures that the trend analysis is instantaneous and does not consume unnecessary computing resources.

Task:

Create a query on the materialized view that generates a report with the following 4 columns:

- » 1. **Date** : The date of sale.
- » 2. **Sales_Today** : The total sold that day.
- » 3. **Sales_Yesterday** : The total sold the previous day (using window functions).
- » 4. **Diff_Percentual** : The percentage of growth or decrease compared to the previous day, rounded to 2 decimal places.



The screenshot shows the Google Cloud BigQuery console interface. At the top, there's a header with the query name "Task S4.01. BigQuery Advance...", a "Run" button, and options for "Save", "Schedule", and "Share". Below the header is a menu bar with "File", "Edit", "Run", and "Tools". The main area displays a SQL query:

```
114 WITH Sales_Trend AS
115 (
116   SELECT
117     date,
118     total_sales_per_day AS Sales_Today,
119     LAG(total_sales_per_day) OVER(ORDER BY date) AS Sales_Yesterday
120   FROM `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`
121 )
122 SELECT
123   date,
124   Sales_Today,
125   Sales_Yesterday,
126   ROUND(
127     SAFE_DIVIDE(
128       Sales_Today - Sales_Yesterday,
129       Sales_Yesterday
130     ) * 100,
131     2
132   ) AS Diff_Percentual
133 FROM Sales_Trend
134 ORDER BY date;
```

Below the query, a status bar indicates: "This script will process 21.56 MB when run." and "Using on-demand processing quota".

The bottom section shows the "Query results" tab. It includes a "Recent" list on the left with a dropdown arrow and a "Query results" header with options for "Chat", "View all jobs", "Save results", and "Open in". The results table has columns: "Row", "date", "Sales_Today", "Sales_Yesterday", and "Diff_Percentual". The first row is highlighted, showing a green checkmark and the query name "WITH Sales_Tren...".

114

115 WITH Sales_Trend AS

116 (

117 SELECT

118 date,

119 total_sales_per_day AS Sales_Today,

120 LAG(total_sales_per_day) OVER(ORDER BY date) AS Sales_Yesterday

✓

This script will process 21.56 MB when run.

Using on-demand processing quota

Recent

Query results

Chat

View all jobs

Save results

Open in

12:14

Job information

Results

Visualisation

JSON

Execution details

Execution graph

✓ WITH Sales_Trend ...

12:13

create materialized ...

12:13

✓ DROP MATERIALIZ...

12:12

12:11

Row	date	Sales_Today	Sales_Yesterday	Diff_Percentual
1	2026-08-17	533175.4100000...	null	null
2	2026-08-18	521165.2800000...	533175.4100000...	-2.25
3	2026-08-19	538803.4400000...	521165.2800000...	3.38
4	2026-08-20	515735.0800000...	538803.4400000...	-4.28
5	2026-08-21	531096.7300000...	515735.0800000...	2.98
6	2026-08-22	506911.9200000...	531096.7300000...	-4.55
7	2026-08-23	525463.2100000...	506911.9200000...	3.66
8	2026-08-24	531955.18	525463.2100000...	1.24
9	2026-08-25	521778.9600000...	531955.18	-1.91
10	2026-08-26	507766.0700000...	521778.9600000...	-1.55

Exercise 3: Running Totals (Running Totals on Views)

Scenario:

The Chief Financial Officer (CFO) needs to visualize the annual growth curve. He is not so interested in what was sold today, but rather how much we have invoiced in total since the beginning of the year to date (YTD - Year To Date).

Task:

Using the materialized view mv_daily_sales(to avoid recalculating base aggregations), generate a report with three columns:

- » 1. **Date** .
- » 2. **Sales of the Day** : Rounded to 2 decimal places.
- » 3. **Cumulative Sales YTD** : A progressive sum of sales that must be restarted every **January 1**. The final result must be rounded to 2 decimal places.

```
127 --» 2. Sales of the Day : Rounded to 2 decimal places.
128 --» 3. Cumulative Sales YTD : A progressive sum of sales that must be restarted every January 1. The final result must
129 be rounded to 2 decimal places.
130
131 select date,
132        round(total_sales_per_day, 2) AS Sales_of_the_Day,
133        round(sum(total_sales_per_day) over(partition by extract(year from date)
134        order by date
135        rows between unbounded preceding AND current ROW), 2) AS Cumulative_Sales_YTD
136 from `sprint3-analytics-mahnaz.sprint3_gold.mv_daily_sales`
137 order by date;
```

✓ This query will process 800 B when run.

Using on-demand processing quota

Query results					Chat	View all jobs	Save results ▾	Open in ▾	⌵	✕
Job information					Results	Visualisation	JSON	Execution details	Execution graph	
Row	date	Sales_of_the_Day	Cumulative_Sales...							
1	2026-08-16	514402.19	514402.19							
2	2026-08-17	524453.96	1038856.15							
3	2026-08-18	516969.35	1555825.5							
4	2026-08-19	513898.03	2069723.53							
5	2026-08-20	525018.78	2594742.31							

Exercise 4: Customer Loyalty and Value (Advanced Filtering)

Scenario:

The Marketing team launches the campaign "The third one is the best". They want to send a gift to users who have completed their third transaction. However, the gift budget is not fixed: it depends on the "quality" of the customer at the beginning. They need to know the Average Ticket of these first 3 purchases to segment whether to send a basic item or a premium gift.

✚ Task:

Generates a list of users who have reached (or exceeded) their third purchase, showing:

- » 1. **User data (user_id, full_name, email) .**
- » 2. **The date and exact amount of the 3rd purchase .**
- » 3. **Average of the first 3 :** The average spend of your transactions 1, 2 and 3.

💡 Technical Hint:

- » **You must use the ROW_NUMBER() function to establish the chronological order of purchases .**
- » **You must use the QUALIFY clause to filter .**
- » **Optimization Challenge :** Find the strategy with the lowest computational cost. Keep in mind that calculating averages over the entire historical record is expensive. Try to filter out the necessary rows before performing the final aggregation.

```
with Customer_third_purchase AS
(
    select user_id, timestamp, amount,
           row_number() over(
                           partition by user_id
                           order by timestamp
                           ) AS purchase_rank
    from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean`
    qualify row_number() over(
                           partition by user_id
                           order by timestamp
                           ) <=3
)
select us.user_id, concat(us.name, ' ', us.surname) AS full_name, us.email,
       ctp.timestamp, ctp.amount,
       round(avg(ctp.amount) over(partition by ctp.user_id), 2) AS avrage_3_purchases

from `sprint3-analytics-mahnaz.sprint3_silver.users_combined` AS us
join Customer_third_purchase AS ctp
on us.user_id=ctp.user_id
qualify purchase_rank=3;
```

Task S4.01. BigQuery Advance...

Run Save Schedule Share

File Edit Run Tools

```

136 order by date;
137
138 -- Exercise 4: Customer Loyalty and Value (Advanced Filtering)
139 -- Generates a list of users who have reached (or exceeded) their third purchase, showing:
140 --» 1. User data (user_id, full_name, email) .
141 --» 2. The date and exact amount of the 3rd purchase .
142 --» 3. Average of the first 3 : The average spend of your transactions 1, 2 and 3.
143
144 with Customer_third_purchase AS
145 (
146   select user_id, timestamp, amount,
147          row_number() over(
148            partition by user_id
149            order by timestamp
150          ) AS purches_rank

```

This query will process 2.27 MB when run.

Using on-demand processing quota

Query results Chat View all jobs Save results Open in

Job information Results Visualisation JSON Execution details Execution graph

Row	full_name	email	timestamp	amount	avrage_3_purchases
1	Amkjrjv Qbulrxbp	amkjrjv.qbulrxbp@example.com	2016-07-10 07:40:56 UTC	136.3	100.0:
2	Nfvrlb Oydaiwbg	nfvrlb.oydaiwbg@example.com	2015-07-07 23:20:48 UTC	245.36	298.:
3	Ijbfmd Jbddzhvp	ijbfmd.jbddzhvp@example.com	2017-06-14 22:53:41 UTC	71.89	127.5:
4	Uyciig Sfbdymzj	uyciig.sfbdymzj@example.com	2017-10-26 09:41:07 UTC	180.27	107.:
5	Yjqurq Ojizvgqi	yjqurq.ojizvgqi@example.com	2016-02-09 12:40:58 UTC	73.31	103.5:

Level 3: Analytics Engineering (Arrays & Automation)

Scenario:

Marketing urgently needs a Ranking of Best-Selling Products.

Exercise 1: Unnesting and Flattening Data (Unnesting)

Scenario:

Currently, in the Transactions table, the products sold are "hidden" inside an Array (a list of IDs). The sales team wants to analyze which specific products sell the most, but they can't do that if the data is compressed into a list. They need a Flat Table where each row represents a product sold, even if that means repeating the transaction ID.

Task:

Create the table `dim_transactions_flat` by denormalizing the information. You need to "explode" the product array and cross-reference it with the master catalog to get the individual names and prices.

Technical Steps:

- » 1. Use `CROSS JOIN UNNEST(product_ids)` to transform the Array into individual rows.
- » 2. Perform a query `JOIN` on the products table to obtain the name and price of each item.
 - 📌 Note: Be careful with data types when making the `JOIN` (Array de `INT64` vs `Product_ID STRING`).
- » 3. Don't group the result. We want to see the breakdown line by line.

```
170 cross-reference it with the master catalog to get the individual names and prices.
171 --» 1. Use CROSS JOIN UNNEST(product_ids) to transform the Array into individual rows.
172 --» 2. Perform a query JOIN on the products table to obtain the name and price of each item.
173 --» 3. Don't group the result. We want to see the breakdown line by line.
174
175 create or replace table `sprint3-analytics-mahnaz.sprint3_gold.dim_transactions_flat` AS
176 (
177   select tr.transaction_id, tr.timestamp, tr.amount, p.product_id as product_sku, p.name as product_name, p.price as
product_price
178   from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean` AS tr
179   cross join unnest(tr.product_ids) AS product_id
180   join `sprint3-analytics-mahnaz.sprint3_silver.products_clean` as p
181   on product_id = SAFE_CAST(p.product_id AS INT64)
182 );
183
184 select *
185 from `sprint3-analytics-mahnaz.sprint3_gold.dim_transactions_flat`
186 order by transaction_id;
187
188
```

🌱 This script will process 21.56 MB when run.

Using on-demand processing quota

Query results [Chat](#) [View all jobs](#) [Save results](#) [Open in](#)

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	transaction_id	timestamp	amount	product_sku	product_name
1	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	87	sith Jade

sprint3-analytics-mahnaz / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti...

QueryChatOpen in + Share Copy

⋮ Refresh

Schema

DetailsPreviewInsightsLineageData profileData Quality

Filter Enter property name or value?

☐	Field name	Type	Mode	Description	Key	Collation	Default value	Data g
☐	transaction_id	STRING	NULLABLE	-	-	-	-	-
☐	timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-
☐	amount	FLOAT	NULLABLE	-	-	-	-	-
☐	product_sku	INTEGER	NULLABLE	-	-	-	-	-
☐	product_name	STRING	NULLABLE	-	-	-	-	-
☐	product_price	FLOAT	NULLABLE	-	-	-	-	-
☐	product_price_tax_inc	FLOAT	NULLABLE	-	-	-	-	-

184 select *
185 from `sprint3-analytics-mahnaz.sprint3\_gold.dim\_transactions\_flat`
186 order by transaction_id;
187
188
189
190

✔ This script will process 21.56 MB when run.
Using on-demand processing quota

Query results

ChatView all jobsSave resultsOpen in

Job information

Results

VisualisationJSONExecution detailsExecution graph

Row	transaction_id	timestamp	amount	product_sku	product_name
1	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	87	sith Jade
2	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	26	Stark Karstark
3	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	16	the duel warden
4	00043A49-2949-494B-A5DD-A5...	2024-08-28 07:16:46 UTC	395.43	97	jinn Winterfell
5	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	69	Littlefinger the giantsblood
6	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	87	sith Jade
7	000447FE-B650-4DCF-85DE-C7...	2016-12-21 20:07:18 UTC	155.63	66	mustafar jinn
8	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	16	the duel warden
9	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	30	Karstark warden
10	00045D6B-ED2E-4F2F-8186-CE...	2020-07-14 15:37:45 UTC	326.01	11	Karstark Dorne

Exercise 2: Sales Ranking (Simple Aggregation)

Scenario:

Now that you have the infrastructure ready and the `dim_transactions_flat` table (where each row equals one product sold) available, generating analytical reports is trivial. You no longer need complex unwrapping functions to answer basic business questions.

Task:

Generates the **Top 5 best-selling products** in the company's history.

Method:

Consult your new denormalized table (`dim_transactions_flat`) directly.

» Being a flat table, you simply need a standard grouping (`GROUP BY`) by the product name and a count (`COUNT`) of the rows.

Delivery:

- » The SQL code that generates the ranking.
- » A screenshot of the result with the 5 winning products and how many units of each were sold.

```
select dt.product_name, count(*) as units_sold
from `sprint3-analytics-mahnaz.sprint3_gold.dim_transactions_flat` as dt
group by dt.product_name
order by units_sold desc
limit 5;
```

```
204
205 --Exercise 2: Sales Ranking (Simple Aggregation)
206 --Generates the Top 5 best-selling products in the company's history.
207 --Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).
208 --» En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte
    (COUNT) de les files.
209
210 select dt.product_name, count(*) as units_sold
211 from `sprint3-analytics-mahnaz.sprint3_gold.dim_transactions_flat` as dt
212 group by dt.product_name
213 order by units_sold desc
214 limit 5;
215
```

✓ Query completed

Using on-demand processing quota

Recent	Query results	Chat	View all jobs	Save results	Open in	
12:46	Job information	Results	Visualisation	JSON	Execution details	Execution graph
✓ select dt.product...	Row	product_name	units_sold			
12:22	1	Winterfell	10071			
✓ with Customer_th...	2	the duel	10059			
12:21	3	dooku solo	10022			
! with Customer_th...	4	Karstark Dorne	7694			
12:14	5	skywalker ewok	7607			
WITH Sales Tran						

Exercise 3: Pipeline Automation and Visualization

Scenario:

The static report from the previous year was very pleasing, but the Marketing Director needs to professionalize it for daily decision-making. He has two urgent requirements:

- » **1. Centralized Logic** : They want to see the price with VAT (21%) broken down. However, since the tax can change by country or year, they prohibit you from putting the calculation $\text{price} * 1.21$ directly in the code ("Hardcoded"). You have to use a reusable function.
- » **2. Data Freshness** : The dashboard should be automatically updated every morning at 07:00 AM, before the daily sales meeting.

Task:

You must evolve your table `dim_transactions_flatto` include tax calculation and automate its regeneration.

Technical Steps:

- » **1. User Defined Functions (UDF)** : Create a persistent SQL function called `calculate_tax(amount)` that receives a numeric value and returns the result applying 21% tax.

```
188
189 --Exercise 3: Pipeline Automation and Visualization
190 --You must evolve your table dim_transactions_flatto include tax calculation and automate its regeneration.
191 --1. User Defined Functions (UDF) : Create a persistent SQL function called calculate_tax(amount) that receives a
192   numeric value and returns the result applying 21% tax.
193 create or replace function `sprint3-analytics-mahnaz.sprint3_gold.calculate_tax`(amount float64)
194   returns float64
195   as(
196     round(
197       (amount* 1.21),2
198     );
199
```

✓ This script will process 21.56 MB when run.

Query results

 Chat

[View all jobs](#)

Save results ▾

Open in ▾



Job information

Results

Execution details

Execution graph



This statement replaced the function named `sprint3-analytics-mahnaz.sprint3_gold.calculate_tax`.

[Go to function](#)

» 2. Integration and Orchestration :

- Modify the table creation code (from Exercise 1) to use your new `calculate_tax` function and generate a new column: `product_price_tax_inc`.
- Configure a Scheduled Query in BigQuery so that this statement (CTAS) runs automatically every day at 07:00 AM.

```
201
202 --2. Integration and Orchestration :
203 --Modify the table creation code (from Exercise 1) to use your new calculate_tax function and generate a new column:
  product_price_tax_inc.
204 --Configure a Scheduled Query in BigQuery so that this statement (CTAS) runs automatically every day at 07:00 AM.
205 create or replace table `sprint3-analytics-mahnaz.sprint3_gold.dim_transactions_flat` AS
206 (
207   select tr.transaction_id, tr.timestamp, tr.amount, p.product_id AS product_sku, p.name as product_name, p.price as
  product_price,
208   `sprint3-analytics-mahnaz.sprint3_gold.calculate_tax`(p.price) AS product_price_tax_inc
209
210   from `sprint3-analytics-mahnaz.sprint3_silver.transactions_clean` AS tr
211   cross join unnest(tr.product_ids) AS product_id
212   join `sprint3-analytics-mahnaz.sprint3_silver.products_clean` as p
213   on product_id = SAFE_CAST(p.product_id AS INT64)
214 )
```

✓ Query completed

Using on-demand processing quota

Query results

Chat

[View all jobs](#)

Save results ▾

Open in ▾



Job information

Results

Execution details

Execution graph

i This statement replaced the table named `dim_transactions_flat`.

[Go to table](#)

Export as scheduled query



Settings valid.

Details and schedule

Name for scheduled query *

Daily_dim_transactions_flat_refresh

Schedule options

Repeat frequency *

Days



At *

7:00

UTC



Start now



Start at set time

Start date and run time

05/10/2026, 12:57

CET



End never



Schedule end time

End date

CET



Scheduled to run every day at 7:00 UTC starting now.

Save

Cancel

Scheduled daily at 07:00 UTC. If the business requirement refers to local Barcelona time, the UTC schedule should be adjusted according to CET/CEST.

3. Visualization (BI) : Connect Looker Studio to your table dim_transactions_flat and create a Dashboard: "Sales Performance Monitor"

👉 Delivery:

<https://datastudio.google.com/reporting/c174d1ac-16f1-4c27-9fc9-086b3b8b5d26>

